

dataset2.py

```
1 import json
2 import torch
3 from PIL import Image
4 from torch.utils.data import Dataset, DataLoader
5 from torchvision import transforms
6
7 from config2 import (
8     IMAGE_SIZE, NORM_STATS_PATH, BATCH_SIZE, NUM_WORKERS,
9     make_generator, seed_worker,
10 )
11
12
13 def load_norm_stats():
14     """Read train-only mean/std saved by data_setup2.py."""
15     try:
16         with open(NORM_STATS_PATH, "r", encoding="utf-8") as f:
17             s = json.load(f)
18             return s["mean"], s["std"]
19     except FileNotFoundError:
20         raise FileNotFoundError(
21             "norm_stats.json not found - run python data_setup2.py first."
22         )
23
24
25 class AddGaussianNoise:
26     """Add zero-mean Gaussian noise to a [0,1] tensor, then clamp. Deterministic
27     under a global torch seed. Applied to TRAIN ONLY."""
28     def __init__(self, std: float = 0.06):
29         self.std = std
30     def __call__(self, t: torch.Tensor) -> torch.Tensor:
31         if self.std <= 0:
32             return t
33         noise = torch.randn_like(t) * self.std
34         return torch.clamp(t + noise, 0.0, 1.0)
35     def __repr__(self):
36         return f"AddGaussianNoise(std={self.std})"
37
38
39 def build_transforms(aug_level: str):
40     """Return (train_transform, eval_transform) for a given augmentation level."""
41     mean, std = load_norm_stats()
42     normalize = transforms.Normalize(mean=[mean], std=[std])
43
44     # Eval / test: CLEAN - no noise, no jitter (per professor)
45     eval_transform = transforms.Compose([
46         transforms.Grayscale(num_output_channels=1),
47         transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
48         transforms.ToTensor(),
49         normalize,
50     ])
51
52     if aug_level == "geometric":
53         train_transform = transforms.Compose([
54             transforms.Grayscale(num_output_channels=1),
55             transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
56             transforms.RandomAffine(degrees=15, translate=(0.1, 0.1)),
57             transforms.RandomHorizontalFlip(p=0.5),
58             transforms.ToTensor(),
59             normalize,
60         ])
61
62     elif aug_level == "robust":
63         train_transform = transforms.Compose([
64             transforms.Grayscale(num_output_channels=1),
65             transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.8, 1.0)),
66             transforms.RandomAffine(degrees=15, translate=(0.1, 0.1)),
67             transforms.RandomHorizontalFlip(p=0.5),
68             transforms.ColorJitter(brightness=0.3, contrast=0.3),
69             transforms.ToTensor(),
70             AddGaussianNoise(std=0.06),
```

```

71         normalize,
72     ])
73
74     elif aug_level == "robust_plus":
75         train_transform = transforms.Compose([
76             transforms.Grayscale(num_output_channels=1),
77             transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.75, 1.0)),
78             transforms.RandomAffine(degrees=20, translate=(0.12, 0.12)),
79             transforms.RandomHorizontalFlip(p=0.5),
80             transforms.ColorJitter(brightness=0.4, contrast=0.4),
81             transforms.ToTensor(),
82             AddGaussianNoise(std=0.08),
83             normalize,
84         ])
85     else:
86         raise ValueError(f"Unknown aug_level: {aug_level}")
87
88     return train_transform, eval_transform
89
90
91 class GrayImageDataset(Dataset):
92     """Loads (image, label) from a dataframe with columns filepath/label."""
93     def __init__(self, df, transform):
94         self.filepaths = df["filepath"].tolist()
95         self.labels = df["label"].tolist()
96         self.transform = transform
97     def __len__(self):
98         return len(self.filepaths)
99     def __getitem__(self, idx):
100         img = Image.open(self.filepaths[idx]).convert("L") # force 1 channel
101         img = self.transform(img)
102         return img, self.labels[idx]
103
104
105 def get_dataloaders(train_df, val_df, aug_level, batch_size=BATCH_SIZE):
106     """Build train (augmented) + val (clean) loaders. Deterministic shuffling."""
107     train_tf, eval_tf = build_transforms(aug_level)
108
109     train_ds = GrayImageDataset(train_df, transform=train_tf)
110     val_ds = GrayImageDataset(val_df, transform=eval_tf)
111
112     g = make_generator()
113     train_loader = DataLoader(
114         train_ds, batch_size=batch_size, shuffle=True,
115         num_workers=NUM_WORKERS, pin_memory=True,
116         worker_init_fn=seed_worker, generator=g, drop_last=False,
117     )
118     val_loader = DataLoader(
119         val_ds, batch_size=batch_size, shuffle=False,
120         num_workers=NUM_WORKERS, pin_memory=True,
121         worker_init_fn=seed_worker, generator=g,
122     )
123     return train_loader, val_loader, eval_tf

```